preserve the property that you don’t have to process every value and thus improve the performance compared to the serial version in more circumstances. The downside to this is that atomic loads can be slow operations, so this can impede the progress of each thread.

Now you have two choices as to how to return the values and how to propagate any exceptions. You can use an array of futures, use `std::packaged_task` for transferring the values and exceptions, and then process the results back in the main thread; or you can use `std::promise` to set the final result directly from the worker threads. It all depends on how you wish to handle exceptions from the worker threads. If you want to stop on the first exception (even if you haven’t processed all elements), you can use `std::promise` to set both the value and the exception. On the other hand, if you want to allow the other workers to keep searching, you can use `std::packaged_task`, store all the exceptions, and then rethrow one of them if a match isn’t found.

In this case I’ve opted to use `std::promise` because the behavior matches that of `std::find` more closely. One thing to watch out for here is the case where the element being searched for isn’t in the supplied range. You therefore need to wait for all the threads to finish *before* getting the result from the future. If you just block on the future, you’ll be waiting forever if the value isn’t there. The result is shown here.

Listing 8.9. An implementation of a parallel find algorithm

```
template<typename Iterator,typename MatchType>
Iterator parallel_find(Iterator first,Iterator last,MatchType match)
{
    struct find_element    <-(1)
    {
    void operator()(Iterator begin,Iterator end,
                    MatchType match,
                    std::promise<Iterator>* result,
                    std::atomic<bool>* done_flag)
    {
        try
        {
            for(;(begin!=end) && !done_flag->load();++begin)    <-(2)
            {
                if(*begin==match)
                {
```